Reflection and Traceability Report on Software Engineering

Team #16, The Chill Guys
Hamzah Rawasia
Sarim Zia
Aidan Froggatt
Swesan Pathmanathan
Burhanuddin Kharodawala

[Reflection is an important component of getting the full benefits from a learning experience. Besides the intrinsic benefits of reflection, this document will be used to help the TAs grade how well your team responded to feedback. Therefore, traceability between Revision 0 and Revision 1 is and important part of the reflection exercise. In addition, several CEAB (Canadian Engineering Accreditation Board) Learning Outcomes (LOs) will be assessed based on your reflections. —TPLT]

1 Changes in Response to Feedback

The subsections below document **capstone peer-review** feedback from GitHub issues. For each item we state the source, a concise summary of the issue, our response (what changed), and traceability links. The main pull request bundling MIS, VnV Plan, VnV Report, and related CI workflow updates is PR #336; its description lists `Closes` for issues #165, #167, #168, #169, #244, and #245 so those issues close when the PR merges to `main`.

[Summarize the changes made over the course of the project in response to feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. —TPLT]

[For those teams with an external supervisor, please highlight how the feedback from the supervisor shaped your project. In particular, you should highlight the supervisor's response to your Rev 0 demonstration to them. —TPLT]

[Version control can make the summary relatively easy, if you used issues and meaningful commits. If you feedback is in an issue, and you responded in the issue tracker, you can point to the issue as part of explaining your changes. If addressing the issue required changes to code or documentation, you can point to the specific commit that made the changes. Although the links are helpful for the details, you should include a label for each item of feedback so that

the reader has an idea of what each item is about without the need to click on everything to find out. —TPLT]

[If you were not organized with your commits, traceability between feedback and commits will not be feasible to capture after the fact. You will instead need to spend time writing down a summary of the changes made in response to each item of feedback. —TPLT]

[You should address EVERY item of feedback. A table or itemized list is recommended. You should record every item of feedback, along with the source of that feedback and the change you made in response to that feedback. The response can be a change to your documentation, code, or development process. The response can also be the reason why no changes were made in response to the feedback. To make this information manageable, you will record the feedback and response separately for each deliverable in the sections that follow. —TPLT]

[If the feedback is general or incomplete, the TA (or instructor) will not be able to grade your response to feedback. In that case your grade on this document, and likely the Revision 1 versions of the other documents will be low. —TPLT]

1.1 SRS and Hazard Analysis

1.2 Design and Design Documentation

The following items address **Module Interface Specification (MIS)** feedback from **peer reviewers** (GitHub issues). We accepted the feedback and updated `docs/Design/SoftDetailedDes` (source `.tex` and rebuilt `MIS.pdf`); revision history in the MIS documents the March 29 update.

#245 — Notation (§4) layout and type definitions. *Source:* Peer review (GitHub issue).
Summary of issue: The primitive-types table was narrow and hard to read; `UpdatedFields` used optional-field "`?`" style notation that did not match how partial updates are described elsewhere.
Response: We expanded the primitive-types table to full text width using `tabularx`; we rewrote `UpdatedFields` in prose so each field may be present or omitted independently, without placeholder syntax.
Traceability: Issue #245; changes in PR #336 (`docs/Design/SoftDetailedDes/sections/04_notation.tex`, `docs/Design/SoftDetailedDes/sections/01_revision_history.tex`).

#244 — Introduction link to the repository. *Source:* Peer review (GitHub issue).
Summary of issue: The MIS should point readers to the implementation repository; the PDF showed only "can be found at" because `\href` was missing its visible-text argument.
Response: We replaced the link with `\url{https://github.com/hitchly/hitchly}` so the URL is both visible and clickable in the PDF.

Traceability: Issue #244; changes in PR #336 (`docs/Design/SoftDetailedDes/sections/03_introduction.tex`, revision history).

1.3 VnV Plan and Report

The following items address **Verification and Validation Plan** and **VnV Report** feedback from **peer reviewers**. We accepted the feedback and updated `docs/VnVPlan`, `docs/VnVReport`, and rebuilt the corresponding PDFs; revision-history rows record the March 29 peer-review pass.

#169 — Recording and evaluation of test results. *Source:* Peer review (GitHub issue).
Summary of issue: The plan did not clearly state where results are stored, how pass/fail is decided, or how defects are tracked.
Response: We added an unnumbered subsection under System Tests (§3) describing storage (CI, manual logs, metrics), pass/fail rules for automated, manual, and inspection-style tests, and defect handling, with a table-of-contents entry so numbered subsections stay aligned.
Traceability: Issue #169; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#165 — Non-functional system tests (performance, security, scalability). *Source:* Peer review (GitHub issue).
Summary of issue: System-level testing for NFRs beyond reliability and usability was thin or missing relative to SRS-style expectations.
Response: In the VnV Plan we added §3.2.3–3.2.5 (performance, security, scalability/capacity) with concrete test cases and tied the §3 introduction to those areas. In the VnV Report we extended §4 with matching evaluation areas and test cases (e.g. latency, transport/config review, mocked auth/payment, concurrent match load).
Traceability: Issue #165; PR #336 (`docs/VnVPlan/VnVPlan.tex`, `docs/VnVReport/sections/04_nonfunctional_requirements.tex`, revision histories).

#167 — Traceability in the VnV Plan. *Source:* Peer review (GitHub issue).
Summary of issue: The plan's requirements-to-tests mapping needed to cover profile/scheduling (FR-2), ratings/reporting (FR-5), and NFR-3–NFR-5 alongside existing rows.
Response: We expanded the §3.3 traceability table with those requirement IDs, notes for recurring/schedule coverage, and pointers to the new NFR subsections.
Traceability: Issue #167; PR #336 (`docs/VnVPlan/VnVPlan.tex`).

#168 — VnV Report trace tables and NFR alignment. *Source:* Peer review (GitHub issue).
Summary of issue: Functional vs. non-functional captions and matrices did not fully align with SRS G.4 wording; FR-2 recurring/schedule testing

and NFR columns were incomplete.
Response: We corrected §9 captions, added a `test-FR2-3` row for recurring-schedule router coverage, and replaced the NFR table with an NFR-1–NFR-5 matrix consistent with the updated plan.
Traceability: Issue #168; PR #336 (`docs/VnVReport/sections/09_trace_to_requirements.tex`, `docs/VnVReport/sections/01_revision_history.tex`).

Repository CI (supporting the same PR). *Source:* Internal team (merge friction on documentation PRs; not a numbered course issue).
Summary of issue: Required GitHub checks stayed on "Expected — Waiting for status" because path-filtered jobs were skipped and, separately, `cancel-in-progress` on Actions cancelled long LaTeX runs when `buildtex` auto-pushed PDFs to the PR branch.
Response: We updated `.github/workflows/ci.yml` so named jobs always report success when paths do not match (no-op steps), fixed the aggregate `CI` job's result logic, and disabled `cancel-in-progress` for CI and `buildtex` to allow runs to finish. Those workflow edits were included on the peer-review PR branch so they reach `main` when PR #336 merges.
Traceability: PR #336 (`.github/workflows/ci.yml`, `.github/workflows/buildtex.yml`).

2 Extras

The team completed two extras that were included in the scope of the project: a **usability testing report** for Hitchly and an **in-app first-use guide** so new users see information about core features when they first open the app after sign-in.

2.1 Usability testing report

We produced a standalone usability testing document under `docs/UsabilityTesting/`. It describes goals, methodology, task scenarios for ten moderated sessions, moderator questions and surveys (including SUS), quantitative metrics (task success, time on task, assists), and results with recommendations. Task design aligns with the functional evaluation areas in the Verification and Validation report so usability findings can be read alongside formal requirement testing. This extra closes the loop between whether the implementation behaves correctly and whether real users can complete the main jobs without unnecessary friction.

2.2 In-app first-use guide (tutorial)

We added a lightweight, role-specific tutorial that appears when a user enters the main app as a **rider** or **driver** for the first time (until they finish or skip it). Content is delivered as a swipeable carousel of short slides covering welcome messaging and how to use primary navigation and key actions

for that role. Completion is stored per user (rider vs. driver tutorial flags) via the backend so the guide does not repeat every launch. Implementation lives under `apps/mobile/features/onboarding/` (e.g. `views/RiderTutorial.tsx`, `views/DriverTutorial.tsx`, shared `components/Carousel.tsx`, and `hooks/useTutorialState.ts`), with the tutorial mounted from the rider and driver app layouts. Together with the written usability report, this in-product guidance supports learnability and reduces reliance on external documentation for first-time use.

3 Design Iteration (LO11 (PrototypeIterate))

3.1 Deployment-driven changes (verification email)

McMaster email verification is a central component of user trust and safety. We initially sent OTP mail over SMTP. After deploying the API to Render, SMTP proved unreliable in that environment (connection and deliverability issues typical of hosted platforms without a dedicated mail relay). We replaced direct SMTP with Brevo's REST API: `packages/emails/client.ts` posts to Brevo's transactional endpoint, and `apps/api/lib/email.ts` wires the client using `BREVO_API_KEY`. The reason for the design change is operational, since verification emails must send consistently in production without fighting the host's network constraints.

3.2 Admin experience and security posture

Early thinking placed admin capabilities inside the mobile app for privileged accounts. That approach risked shipping a hidden surface inside a consumer binary and limited what operators could see. The final design moves administration to a standalone web portal (`apps/admin`) backed by dedicated admin server modules (`apps/api/trpc/routers/admin/`), with dashboard areas for users, safety, operations, health, finances, and related views. The reason for the change is the richer operational insight (activity, logs, infrastructure-style summaries as implemented) and alignment with common practice, where administration is not a secret mode in the rider/driver app.

3.3 Live rides: location and Google Maps

Static match-time estimates were not enough once trips run in the real world. We expanded Google Maps–backed behaviour on the server for distance and ETAs during active trips. The `location` router adds `getTripDriverLiveLocation` (`apps/api/trpc/routers/location.ts`), returning the driver's live coordinates, distance/ETA toward pickup or drop-off depending on request status. The rider experience was updated under `apps/mobile/features/trips/` so live driver location, freshness, and external maps are visible during a ride. Mobile platform configuration (`apps/mobile/app.config.ts`) was adjusted for location permission copy and background behaviour so in-trip tracking matches store policies.

Together, this addresses the client need for transparency while a ride is underway, not only at booking time.

3.4 Mobile configuration in TypeScript

We moved mobile app configuration from a JSON manifest to a typed module (`apps/mobile/app.config.ts`). That change was driven by iteration, as permissions, icons, and Google service wiring grew, a typed config reduced silent mistakes and made updates easier to review alongside code changes.

3.5 Iteration from usability findings

Formal usability testing is documented under `docs/UsabilityTesting/`. Sessions highlighted friction around campus drop-off choice, first-time to/from campus wording, cost visibility on match cards, and discovery of review and report entry points—see results and recommendation tables in that report. In response we carried a post-study interface pass to align visual patterns with familiar rideshare products where appropriate, tighten colour consistency, and improve labels and hierarchy on flows that had caused hesitation. We also shipped the in-app rider/driver tutorials summarized in Section 2 (Extras) of this document so first-time users get in-product guidance instead of inferring every primary action from a cold start. Those changes complement fixes tracked from the usability tables.

4 Design Decisions (LO12)

This section covers **constraints**, **assumptions**, and **limitations**, then summarizes how those factors justify the main architectural and product choices.

4.1 Constraints

Community and platform scope. Hitchly is scoped to the McMaster community with email-domain verification, and to modern iOS and Android clients. That constraint drove a mobile-first product with server-enforced affiliation checks rather than an open web signup.

Privacy and payments. Canadian privacy expectations (e.g. PIPEDA, as reflected in the SRS) and card-data rules (PCI DSS) pushed us toward minimal sensitive storage and payment flows that delegate card handling to Stripe instead of persisting raw payment credentials in our database.

Engineering standards. The SRS calls for TypeScript with linting and formatting enforced in CI. That constraint reinforced a typed codebase end to end and predictable review gates, which mattered for a multi-contributor capstone schedule.

Time, team size, and hosting. A fixed academic term and small team constrained how much custom infrastructure we could own. Deploying the API on

a managed host (e.g. Render) with a managed PostgreSQL provider (e.g. Neon) traded operational control for reliability and speed of iteration. In practice, that hosting model also constrained outbound email, as direct SMTP was unreliable from the deployment environment, which motivated the move to a transactional email API as mentioned in the previous section.

4.2 Assumptions

User conduct and honesty. We assume users act in good faith during trips and that a McMaster-affiliated email (plus driver-license capture for drivers) is an acceptable affiliation signal for a campus pilot, not a substitute for commercial background screening.
Legal compliance by drivers. We assume drivers meet Ontario licensing, insurance, and vehicle rules as stated in the SRS narrative; the app prompts and records verification artifacts but does not replace government or insurer checks.
Technical environment. We assume participants run a current Expo build, stable network access, and that third-party services (maps, email, payments) remain available. We also assume campus and urban travel dominates, so Maps-backed distance, routing, and landmark-style drop-offs remain usable, and rural edge cases are weaker.
Operations. We assume few trusted operators use admin tooling, which supports a separate admin surface with role-appropriate access rather than embedding admin features in consumer apps.
These assumptions justified lighter onboarding than a commercial rideshare platform while still documenting trust and safety paths (reports, reviews, admin visibility).

4.3 Limitations

Verification depth. We do not run institutional background checks or insurance verification beyond license upload and email proof, and risk remains with users and institutional policy, as noted in the SRS limitations.
Dependence on third parties. Maps, email, hosting, and payment providers introduce cost, quota, and outage risk, and behaviour degrades if an API is throttled or unavailable.
Quality and coverage. Automated and manual testing are substantial but not exhaustive under course time pressure, and the primary UI language is English unless otherwise specified in the product.

4.4 Principal architectural and product choices

Given the constraints above, we adopted: tRPC with shared types across mobile, API, and admin to reduce contract drift in a small team; a pnpm monorepo with Turbo so shared packages (database access, email client, config) stay in one repository; PostgreSQL for relational integrity on users, trips, and matches;

and a standalone admin web application separate from rider/driver clients for security, clearer roles, and richer operational views. These choices follow from the need for maintainability, type safety, and clear trust boundaries rather than from novelty for its own sake.

5 Economic Considerations (LO23)

Hitchly is software service mobile application aimed at a bounded campus market.

5.1 Market need

There is a plausible niche market among McMaster-affiliated commuters, which consists of riders who want lower-cost trips than solo driving, public transit, or occasional rideshare apps such as Uber or Lyft, and drivers who want to defray parking, fuel, and wear on recurring routes. A campus-scoped network can emphasize trust (shared institution) and repeat patterns (to/from campus) in ways that differ from general-purpose rideshare for ad-hoc city travel.

5.2 Marketing and adoption

Hitchly would be marketed through channels that already reach commuters: student unions and clubs, faculty and staff associations, sustainability and transportation offices, residence orientation, digital signage, and partnerships with a few high-commute programs as a pilot cohort. The message combines cost sharing with verified McMaster affiliation (email-domain verification). User acquisition is organic and institutional through downloads, verified sign-ups, and completed trips measure traction.

5.3 Cost to build and run

One-time / sunk cost is primarily engineering time to reach a pilot-ready mobile client, API, payments, and operations tooling (capstone-scale effort). Ongoing operating cost scales with usage and includes managed API and database hosting (e.g. Render and Neon-style Postgres), transactional email (e.g. Brevo), Google Maps usage (distance, directions, and in-trip location workflows), Stripe processing and Connect-related fees, mobile store accounts, and domain/DNS. Exact monthly cost depends on trip volume, API call patterns, and chosen service tiers. The team would budget using provider dashboards once traffic is non-trivial.

5.4 Pricing and revenue model

Riders and drivers obtain a fare estimate derived from trip distance and time, and the implementation records a platform fee as a percentage of the charged amount. The deployed configuration applies a 15% platform fee, with the

remainder intended for the driver after Stripe processing. Alternatives for a broader rollout include a university-paid deployment (no per-trip fee for users), a lower take rate for a pilot, or a subscription for verified power users, but the current model favours usage-based revenue tied to completed trips.

5.5 Illustrative break-even framing

Because Hitchly is not sold as discrete units, "how many units to make money" maps to how much trip volume (or institutional funding) covers monthly OPEX. Illustrative only suppose combined hosting, email, maps, and fixed third-party overhead were on the order of a few hundred Canadian dollars per month at pilot scale, and suppose an average completed trip produced roughly $1.50 in platform revenue after Stripe (15% of a modest $10 trip, ignoring Stripe's own fee in this sketch). Then on the order of 200–400 completed trips per month would approach that overhead band. This is not a forecast, only a check that revenue is volume-driven and sensitive to Maps and payment fees. A formal business plan would replace these round numbers with measured usage and provider invoices.

5.6 Potential user base

McMaster's public institutional reporting places total student headcount in the mid-30,000s in recent years, with additional faculty and staff. Not everyone commutes by car or needs rideshare, so the addressable subset is smaller, commuters, occasional riders, and drivers with spare seats, but the ceiling for a McMaster-only product is still in the tens of thousands of affiliated people. Growth beyond that would require a different product scope (multi-campus or public launch), which is outside the current scope and focus of the project.

6 Reflection on Project Management (LO24)

[This question focuses on processes and tools used for project management. —TPLT]

6.1 How Does Your Project Management Compare to Your Development Plan

[Did you follow your Development plan, with respect to the team meeting plan, team communication plan, team member roles and workflow plan. Did you use the technology you planned on using? —TPLT]

6.2 What Went Well?

[What went well for your project management in terms of processes and technology? —TPLT]

6.3 What Went Wrong?

[What went wrong in terms of processes and technology? —TPLT]

6.4 What Would you Do Differently Next Time?

[What will you do differently for your next project? —TPLT]

7 Ethics and Equity (LO18)

[Describe how your team applied the principle of equity and universal design to ensure equitable treatment of all stakeholders. —TPLT]

8 Reflection on Capstone

[This question focuses on what you learned during the course of the capstone project. —TPLT]

8.1 Which Courses Were Relevant

[Which of the courses you have taken were relevant for the capstone project? —TPLT]

8.2 Knowledge/Skills Outside of Courses

[What skills/knowledge did you need to acquire for your capstone project that was outside of the courses you took? —TPLT]